

Confidencialidad: Cifrado Simétrico y Cifrado Asimétrico

Práctica 2.1 - Presencial

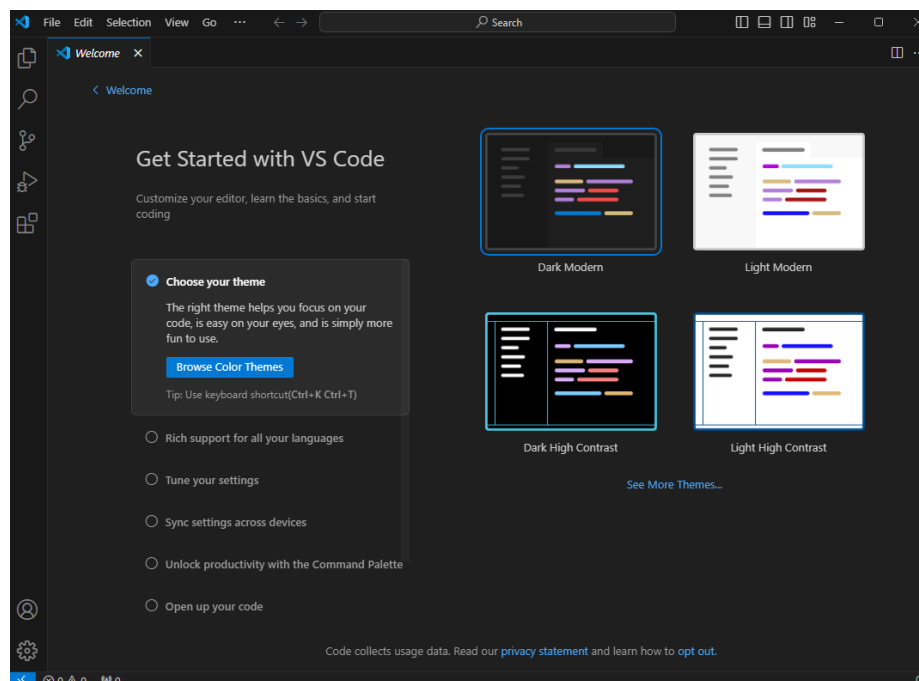
1. Objetivo

Se presenta un doble objetivo en esta parte de la práctica. En primer lugar, **desplegar el entorno de desarrollo** que se utilizará atendiendo a **Visual Studio Code** y lenguaje **Python**. Por otro lado, introduciremos los **conceptos básicos del cifrado simétrico** desde el punto de vista de la **programación** para comprender mejor este tipo de cifrado.

2. Entorno de Desarrollo

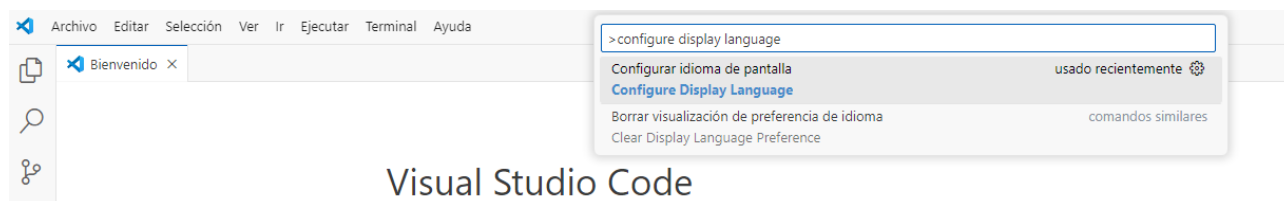
Necesitamos un entorno de desarrollo para llevar a cabo las tareas requeridas en esta parte de la práctica. Para ello, instalaremos en la Máquina Virtual Windows 11 el software Visual Studio Code (VSC). Accede a <https://code.visualstudio.com/download> para poder descargarlo y proceder con la instalación. Si utilizas el ordenador de clase para realizar esta parte, no necesitarás instalar VSC.

Descarga la versión para Windows 11 y procede a realizar la instalación en la Máquina Virtual realizando una instalación típica dejando por defecto todas las opciones que nos muestra el instalador. Una vez instalado, comprueba que se abre de forma correcta y tenemos el software disponible para ser usado.

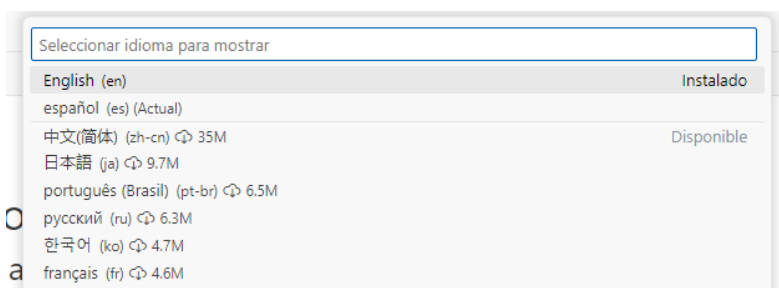


Realiza la configuración del entorno eligiendo el tema del entorno que más se adecúe a tu estilo de programación, así como el idioma en el que lo quieres (en el caso de las prácticas siempre se usarán elementos en español).

Si quieres poner en software en español, una vez abierto presiona la tecla F1 del sistema para abrir la paleta de comandos y escribe “Configure display language”



Al pinchar sobre la opción que nos muestra se mostrarán los idiomas instalados y disponibles. En mi caso ya tengo instalado el español como idioma predeterminado por lo que me aparece en la parte superior. En caso de que no lo tengas instalado deberás proceder a seleccionarlo (se descarga e instala casi instantáneamente).



Con esto tendremos ya configurado en software para desarrollo atendiendo a las características propias del mismo.

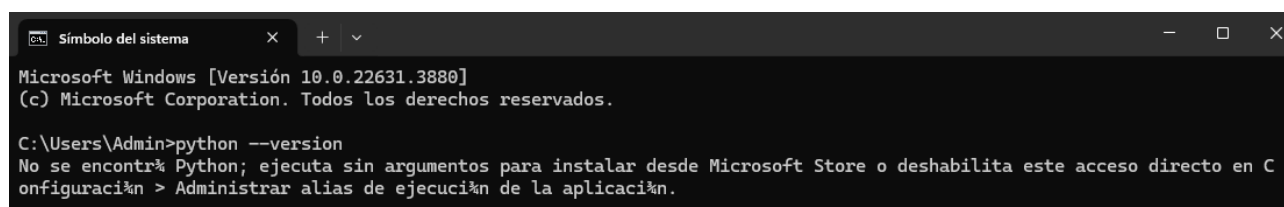
2.1 Instalación y Configuración de VSC para Python

Una vez que tenemos Visual Studio Code (VSC) instalado, tendremos que proceder a realizar la instalación y configuración de VSC para poder utilizar en lenguaje Python.

En primer lugar realizaremos la validación de la versión e instalación de Python. En este caso, y como el desarrollo de Python 2 se interrumpió en 2020, haremos uso de Python 3.

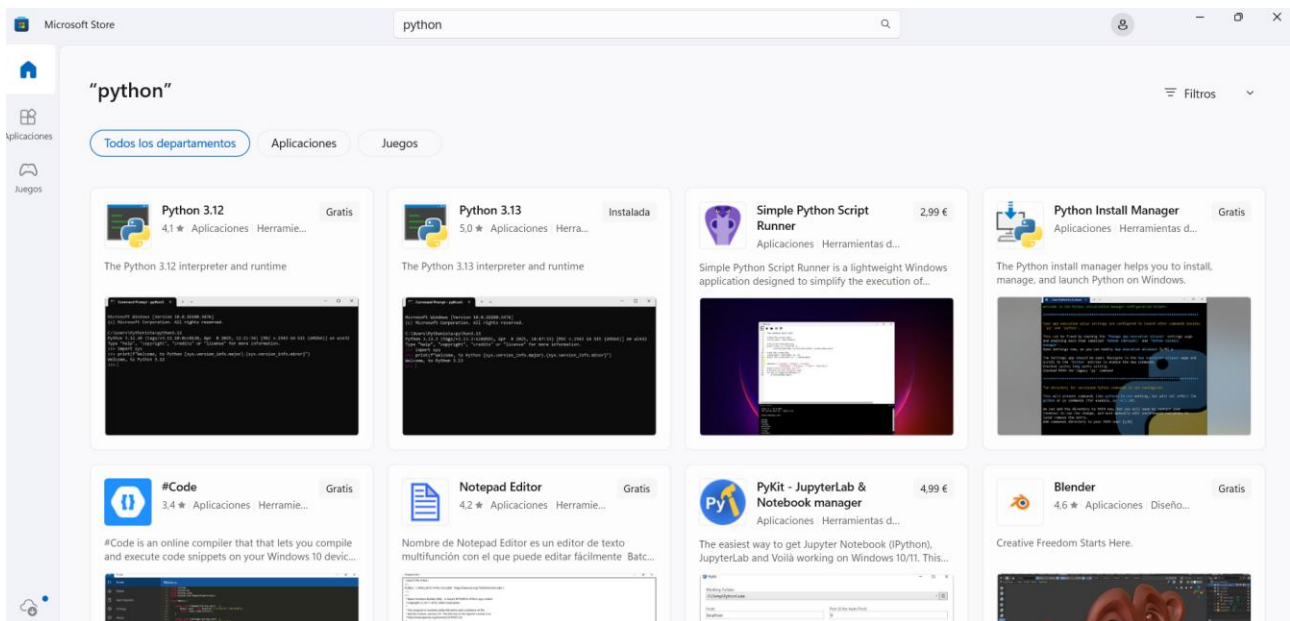
Nota: Algunos sistemas podrían tener preinstalado Python 2, por lo que, deberemos actualizar dicha versión a Python 3.

Para saber si tengo Python 3 instalado en el sistema (no debería porque la instalación se ha realizado desde 0 y no se ha instalado nada), podemos utilizar el símbolo de sistema. Para ello, desde la barra de tareas de Windows pincha en el botón de *Inicio* y posteriormente escribe *cmd*. Abre el símbolo de sistema y teclea el siguiente comando: `python --version` o `py -version`.

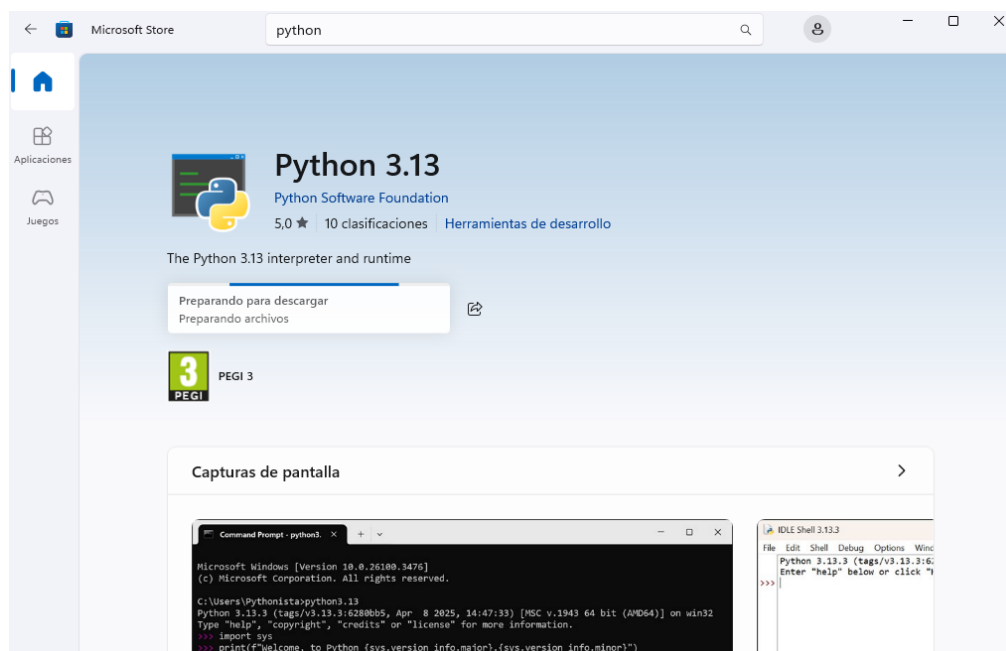


Como podemos observar no tenemos instalado Python en el sistema por lo que, habrá que proceder a su instalación.

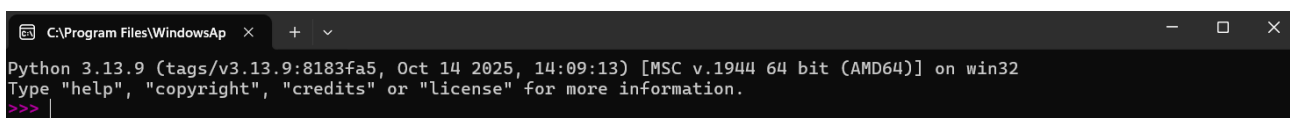
Procederemos a la instalación de Python desde la Microsoft Store. Para ello, desde la barra de tareas, busca la tienda de Microsoft y ábrela. A continuación, busca la palabra Python para poder verificar las diferentes versiones disponibles en la tienda.



Selecciona la versión del lenguaje de programación Python 3.13 y pulsa sobre descargar para que comience la descarga. Una vez finalizada verás que está instalada en el sistema como aparece en la imagen superior.



Una vez instalado, puede que nos aparezca un menú emergente en la parte inferior derecha de la pantalla proponiendo el Inicio de Python. Si pulsamos en *Iniciar* se abrirá una pantalla modo consola como la siguiente:



Si no aparece, puedes acceder desde el menú de inicio al software de Python para poder ejecutar la consola de igual forma. En este consola vemos la versión instalada de Python (3.13.9) y en la que podemos ejecutar comandos en Python. Para comprobar la funcionalidad incluye ejecuta la siguiente sentencia, que imprimirá “Hola Mundo” por pantalla: `print("Hola Mundo")`.

```
C:\Program Files\WindowsAp x + v
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hola Mundo")
Hola Mundo
>>> |
```

Aunque ya hemos comprobado la versión al iniciar Python, accede de nuevo al símbolo de sistema del equipo y ejecuta la orden: `python --version` para comprobar que la versión se corresponde con la que hemos visto anteriormente.

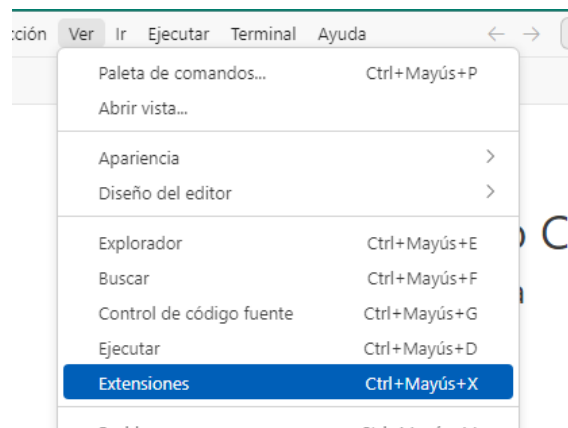
```
Símbolo del sistema x + v
Microsoft Windows [Versión 10.0.26100.6899]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Jose>python --version
Python 3.13.9

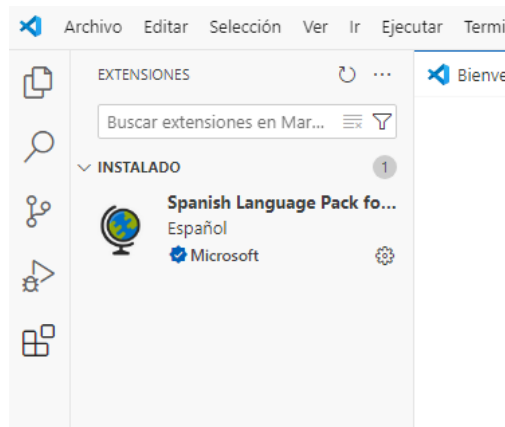
C:\Users\Jose>|
```

Una vez comprobado, ya tendremos instalado correctamente Python en el sistema operativo Windows 11, lo que tendremos que realizar ahora es vincular VSC con Python para poder desarrollar en desde el software instalado.

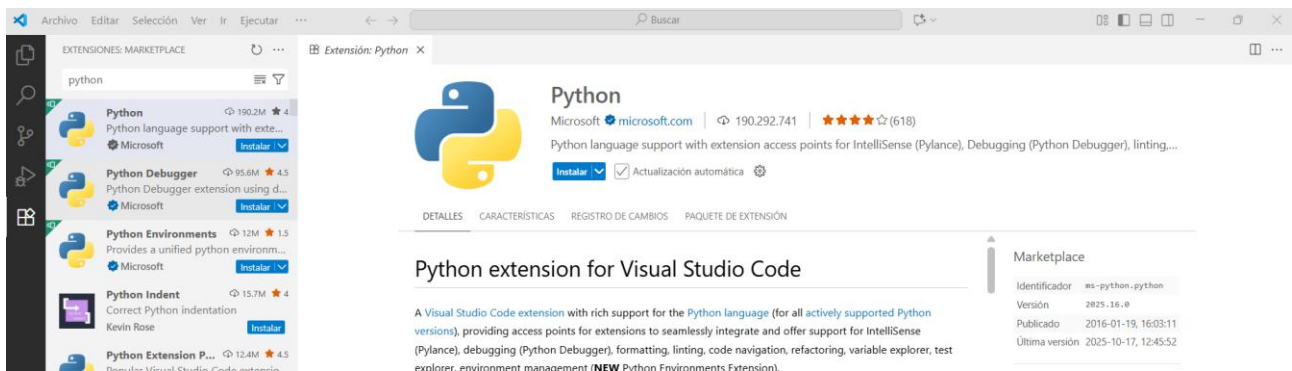
Para realizar esto, habrá que instalar la extensión de Python en VSC. Para ello, abre VSC y ve al menú *Ver → Extensiones*



En la parte izquierda de la pantalla nos aparecerán todas las extensiones, ya instaladas y recomendadas más populares en el *Marketplace*. En mi caso, tengo ya instalada la extensión de paquete de idioma *Español* que instalamos anteriormente.

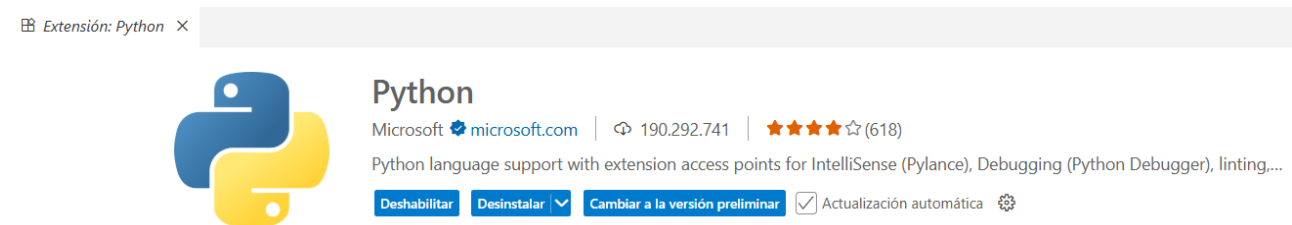


En el buscador escribe *Python* para poder realizar la búsqueda de las extensiones que contengan Python para poder ser usado:

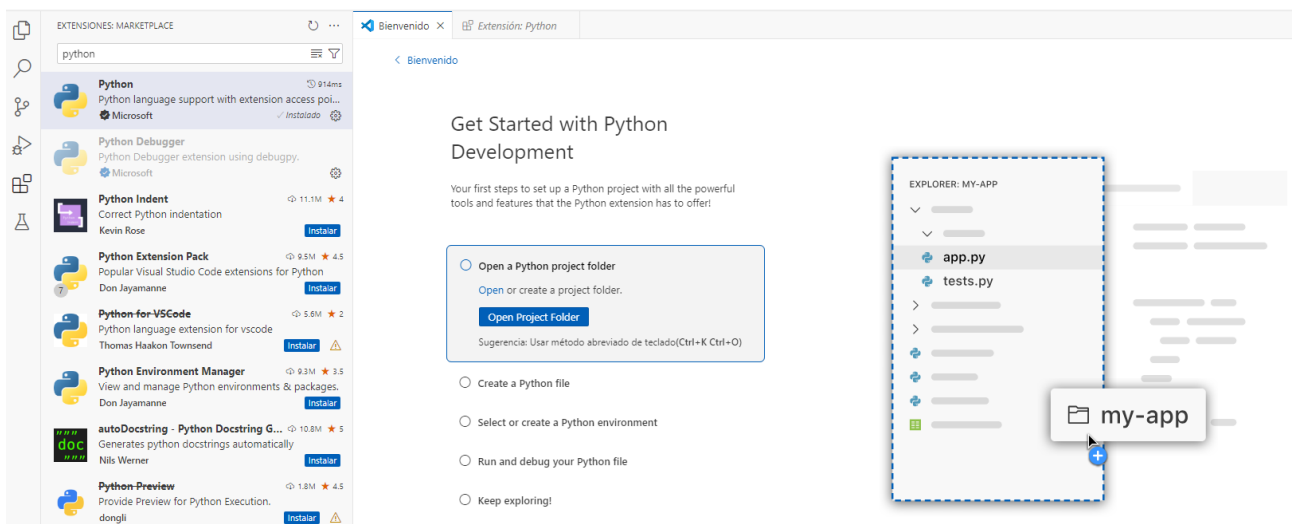


Selecciona la primera de ellas y pulsa sobre el botón instalar que se ve en la descripción de la misma (parte derecha de la pantalla) para proceder a su instalación.

Una vez instalada podremos observar que las opciones de la extensión cambia y, al estar ya instalada, aparecen las opciones de desinstalar y de cambiar a la presión preliminar de la misma. Deja marcada la opción de *Actualización automática* para que si entra una nueva versión nos la instale de forma automática.



La otra pestaña que nos muestra VSC cuando realizamos la instalación los permitirá realizar algunas acciones iniciales sobre la extensión instalada:



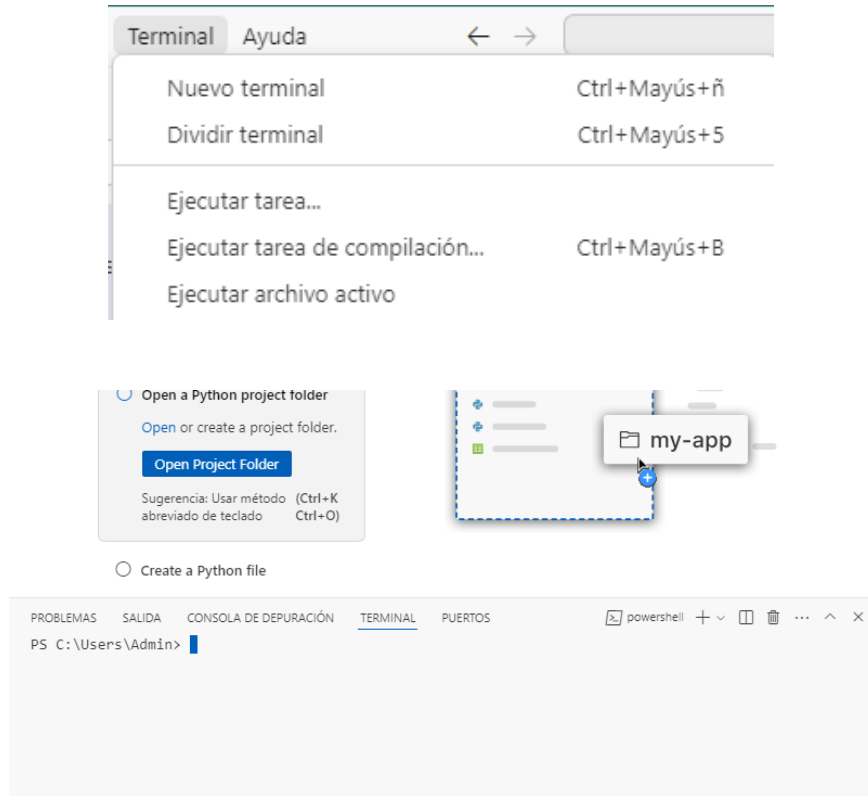
Como vemos, podemos abrir un proyecto en Python, crear uno nuevo o seleccionar un entorno de desarrollo entre otros.

Con esto, ya tendríamos instalado y configurado VSC con la extensión de Python para poder ser usada ya que, previamente, hemos realizado la instalación de Python en nuestro sistema local.

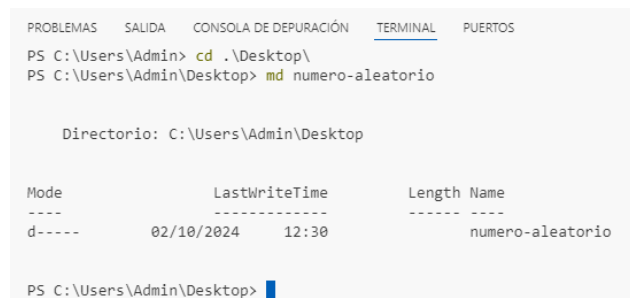
2.2 Primera Aplicación Python – Contraseña Segura

Para crear la primera aplicación, utilizaremos íntegramente VSC para realizarla, aunque hay pasos que se podrían hacer desde fuera del software como la creación de la carpeta que contendrá la aplicación en sí.

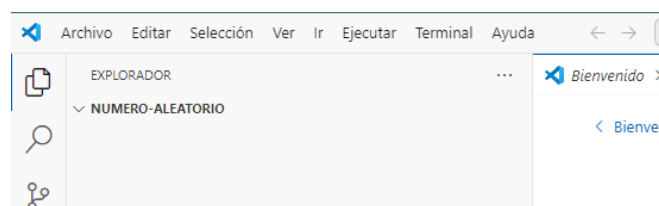
Para ello, abre una nueva terminal dentro de VSC desde el menú *Terminal* → *Nuevo Terminal*. Esto abrirá una terminal en PowerShell que nos permitirá ejecutar comandos.



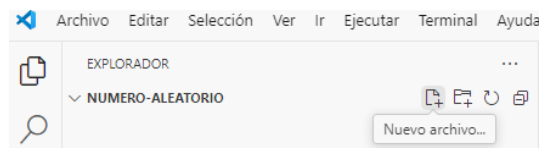
Desde la terminal crea una nueva carpeta en el Escritorio del equipo llamada **numero-aleatorio** para almacenar el código de la aplicación que vamos a desarrollar.



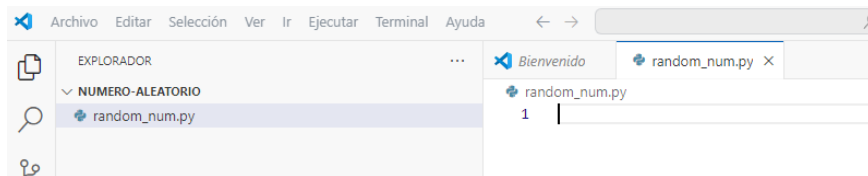
Abre esta carpeta en VSC desde el menú *Archivo* → *Abrir Carpeta* y asegúrate que se abre de forma correcta:



Como puedes ver, todavía no contiene nada, por lo que habrá que crear un nuevo fichero .py que almacenará la lógica de nuestra aplicación.



Pulsa sobre el botón *Nuevo archivo...* para crear un archivo cuyo nombre será: *random_num.py* y guarda dicho fichero.



Ahora estaremos en disposición de escribir el código necesario para llevar a cabo la aplicación que genere números aleatorios en Python.

Llegados a este punto, vamos a comenzar a crear la aplicación para la generación de números aleatorios.

En este caso, Python nos permite usar diferentes métodos para crear elementos aleatorios. Entre ellos, encontramos los siguientes:

- `randint()`: devuelve un número entero comprendido entre los valores indicados. Los valores de los límites inferior y superior también pueden aparecer entre los valores devueltos. Para números decimales (float) se usa la función `uniform()`.
- `randrange()`: devuelve números enteros comprendidos entre un valor inicial y otro final, separados por un valor "paso" determinado.
- `choice()` y `choices()`, permiten seleccionar valores de una lista de forma aleatoria. Toman una lista como argumento y seleccionan aleatoriamente un valor (o valores en el caso de `choices()`).
- `shuffle()`: "baraja" una lista. Esta función "mezcla" o cambia aleatoriamente el orden de los elementos de una lista antes de seleccionar uno de ellos.
- `gauss()`: genera un conjunto de números aleatorios cuya distribución de probabilidad es una distribución gaussiana o normal.

Como **ejemplo** de uso, vamos a crear una pequeña aplicación que determine de forma aleatoria quien comienza a jugar una partida en la que se debe jugar con un dado. Dispondremos de 2 jugadores (Jugador 1 y Jugador 2) y de un dado con 6 caras (del 1 al 6). El programa deberá determinar quién comienza a jugar y qué número saca en su tirada de dado.

Para llevar a cabo este ejemplo utilizaremos `randint()` 2 veces, una para determinar el jugador y otro para determinar la jugada. Verifica el código siguiente para ver lo que se realiza en el ejemplo.

```
import random

#¿Comienza jugador 1 o 2?
start = random.randint(0, 1)
if start == 0:
    print("Empieza el Jugador 1")
else:
    print("Empieza el jugador 2")

#Numero dado
numero_dado = random.randint(1,6)
print ("El número del dado es:", numero_dado)
```


En primer lugar se importa la librería **random** de Python para poder hacer uso de los métodos que contiene para poder generar números aleatorios. A continuación, declaramos una variable *start* que determinará quien empieza a jugar generando un valor entre 0 (Jugador 1) o 1 (Jugador 2). Imprimiremos el resultado atendiendo a una estructura *if*. Por último volvemos a generar un número aleatorio, en este caso entre 1 y 6 para ver la jugada del dado imprimiendo el resultado por pantalla.

Para la ejecución del programa, accede a la terminal y desde ahí introduce el comando : `python nombre_del_ejecutable` como se muestra en la figura:

```
PS C:\Users\Admin\Desktop\número-aleatorio> python .\random_num.py
Comenzamos en 0
El número del dado es: 4
PS C:\Users\Admin\Desktop\número-aleatorio> python .\random_num.py
Empieza el Jugador 1
El número del dado es: 3
PS C:\Users\Admin\Desktop\número-aleatorio> python .\random_num.py
Empieza el Jugador 1
El número del dado es: 6
PS C:\Users\Admin\Desktop\número-aleatorio> █
```

Puedes observar que se ejecutó varias veces para verificar el funcionamiento del número aleatorio en este caso. También puedes ejecutar el ejercicio pulsando el botón “Play” de la parte superior derecha del entorno de desarrollo.

Como **ejercicio final**, y haciendo uso de los métodos explicados anteriormente, se propone la creación de un programa (*Contra_Segura.py*) que sea capaz de crear contraseñas seguras de forma aleatoria. En este caso las contraseñas deben cumplir lo siguiente:

- Tener al inicio 3 números
- A continuación, contener letras (un total de 5 letras mayúsculas y minúsculas)
- Para finalizar, debe contener algún carácter especial de entre los siguientes: *, -, ¿, ? o /
- La longitud final de la contraseña debe ser de 10 caracteres.

Se recomienda incorporar el módulo **string** en la solución para poder hacer uso de los métodos `string.ascii_uppercase` y `string.ascii_lowercase` que nos permitirán abarcar todas las posibilidades existentes en relación a las letras del abecedario tanto en mayúscula como en minúscula.

Puedes ayudarte de una variable llamada *password* para ir almacenando los caracteres creados de forma aleatoria en cada caso con el objetivo de poder concatenarlos y tener la contraseña segura creada. Se recomienda utilizar diferentes métodos de generación de caracteres aleatorios para cada uno de los puntos mencionados anteriormente (números, letras y caracteres especiales).

A modo de ejemplo, se muestra una posible ejecución del programa para verificar que las contraseñas se crean atendiendo a lo especificado en el enunciado del ejercicio:

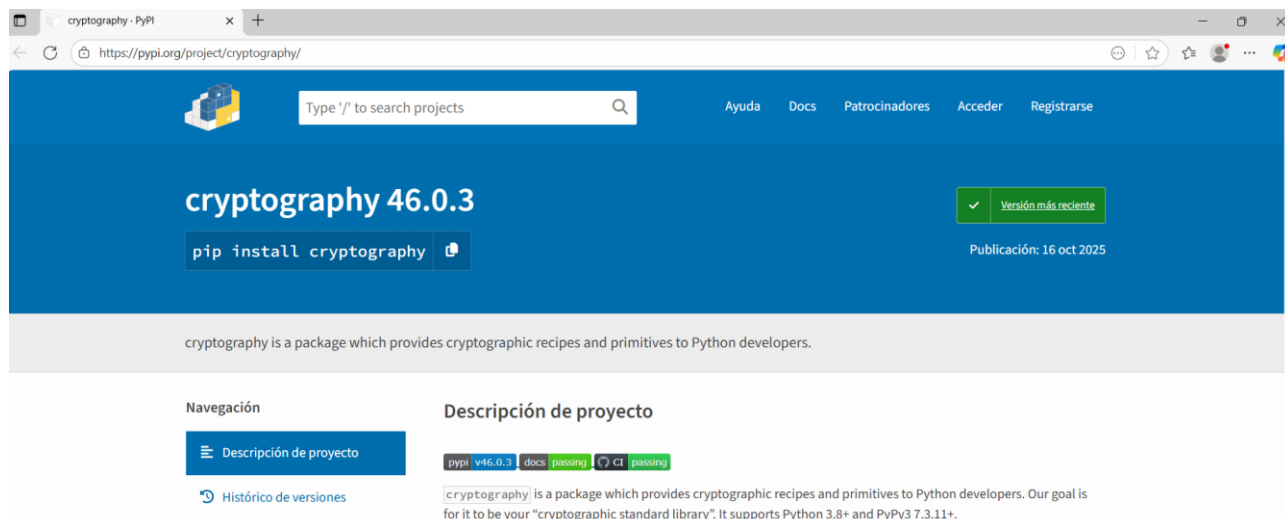
```
PS C:\Users\Admin\Desktop\número-aleatorio> python .\contra_segura.py
Contraseña: 959GZbYo??
PS C:\Users\Admin\Desktop\número-aleatorio> python .\contra_segura.py
Contraseña: 373GwAej¿-
PS C:\Users\Admin\Desktop\número-aleatorio> python .\contra_segura.py
Contraseña: 153gzAat-¿
PS C:\Users\Admin\Desktop\número-aleatorio> python .\contra_segura.py
Contraseña: 986eYvpJ/-
PS C:\Users\Admin\Desktop\número-aleatorio> python .\contra_segura.py
Contraseña: 514CKuiT--
PS C:\Users\Admin\Desktop\número-aleatorio> python .\contra_segura.py
Contraseña: 100uoOkd*/
PS C:\Users\Admin\Desktop\número-aleatorio> python .\contra_segura.py
Contraseña: 552CQZRL?-
```

Merece la pena mencionar que este tipo de contraseña puede ser más robusta si intercambiamos las posiciones de los elementos, pero este ejercicio se propone de esta forma para poder utilizar diferentes métodos del módulo **random** en el desarrollo del mismo.

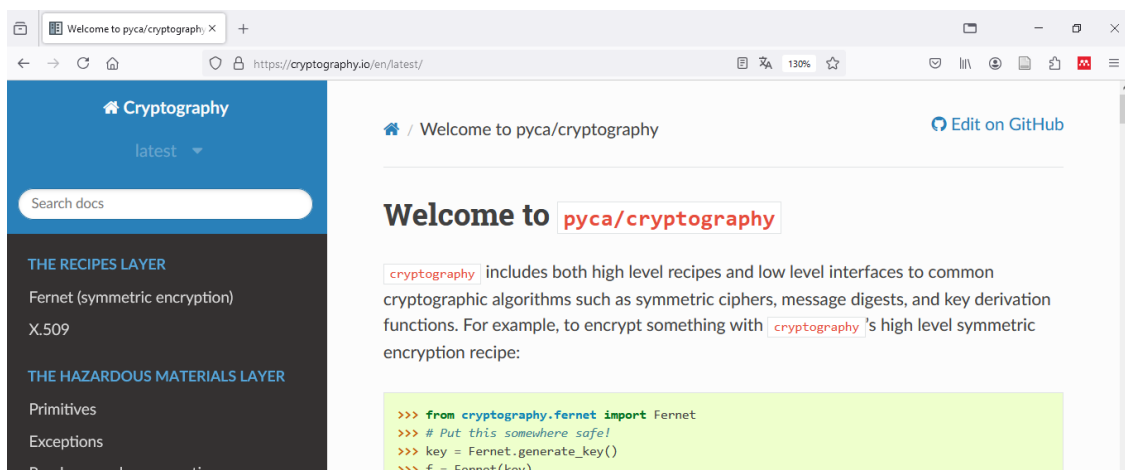
Una vez que ya tenemos nuestra primera aplicación funcionando, y hemos obtenido los conocimientos básicos de Python y conocemos también el entorno de desarrollo que usaremos, nos adentraremos en puntos posteriores en las librerías o módulos que otorgan funcionalidad relativa a la seguridad informática para, por ejemplo, poder hacer cifrados de datos.

2.3 Introducción a Python en Seguridad

Para añadir funcionalidad de elementos relacionados con la Seguridad, usaremos la librería **cryptography**. Esta librería nos va a ofrecer funcionalidad relacionada con criptografía empleados en esta práctica. Accede a <https://pypi.org/project/cryptography/> y echa un vistazo a la introducción que se nos da de este proyecto.

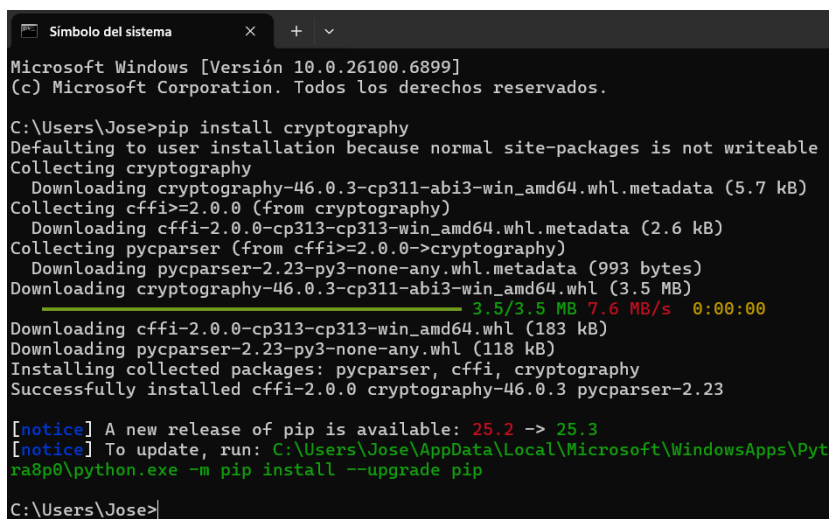


También puedes acceder a la documentación sobre la librería en: <https://cryptography.io/en/latest/>. Aquí encontrarás la última versión disponible y documentación relacionada con el uso de la misma.



Como puedes observar, lo primero que debemos hacer es instalar la librería **cryptography** en el equipo para poder hacer uso de la misma. Para ello, introduce el siguiente comando: `pip install cryptography` en el *Símbolo de Sistema* del equipo.

Verifica que la instalación se realiza de forma correcta en la última versión disponible (en el caso de la creación de este guión de práctica la versión de la librería es la 43.0.1, que se puede observar al final de la ejecución del comando propuesto) que debe coincidir con la que muestra la web del proyecto (visible más arriba).



```

Microsoft Windows [Versión 10.0.26100.6899]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Jose>pip install cryptography
Defaulting to user installation because normal site-packages is not writeable
Collecting cryptography
  Downloading cryptography-46.0.3-cp311-abi3-win_amd64.whl.metadata (5.7 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp313-cp313-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-2.23-py3-none-any.whl.metadata (993 bytes)
Downloaded cryptography-46.0.3-cp311-abi3-win_amd64.whl (3.5 MB)
  3.5/3.5 MB 7.6 MB/s 0:00:00
Downloaded cffi-2.0.0-cp313-cp313-win_amd64.whl (183 kB)
Downloaded pycparser-2.23-py3-none-any.whl (118 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-46.0.3 pycparser-2.23

[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: C:\Users\Jose\AppData\Local\Microsoft\WindowsApps\Python39\python.exe -m pip install --upgrade pip

C:\Users\Jose>

```

Una vez instalada, ya tendremos preparado el entorno para comenzar a crear pequeños programas que hagan uso de elementos criptográficos (encriptación y desencriptación) de información atendiendo a los métodos que la librería nos ofrece.

Una vez concluida esta parte y puesta en marcha nuestra primera aplicación en Python puedes refrescar contenidos asociados con la programación en este lenguaje en la siguiente página web: <https://www.w3schools.com/python/>. Ten en cuenta que trabajaremos en este lenguaje a través de diferentes funcionalidades que ofrece en el mismo, por ejemplo: importando librerías/módulos, creando funciones, manejando ficheros, etc., materia que se da por supuesta para continuar las prácticas.

3. Cifrado Simétrico

Ahora estamos en disposición de comenzar con la creación de dos pequeños programas que utilice Cifrado Simétrico (cifrado y descifrado) para realizar la encriptación y desencriptación de información. Para ello, utilizaremos un **algoritmo de tipo AES** (*Advanced Encryption Standard*) que generará una clave (en un archivo) a través de la cual podremos encriptar nuestra información. Al ser un algoritmo simétrico, utilizaremos la misma clave para desencriptar dicha información.

Importa **Fernet** desde la librería de criptografía que incorporamos anteriormente. Fernet es una herramienta muy útil para desarrollar aspectos de ciberseguridad en Python. Es un pequeño script de Python que se utiliza para resolver problemas comunes y que nos **proporciona cifrado simétrico y autenticación de datos**. Usa AES en modo de operación CBC (*Cipher-Block Chaining*) con una clave de 128 bits. Forma parte de la biblioteca de criptografía para Python, desarrollada por la Autoridad Criptográfica de Python (PYCA). Fernet garantiza que un mensaje cifrado no pueda ser manipulado o leído sin clave.

3.1 Cifrado/Descifrado de mensaje de texto

Crea un directorio llamado *cifra_descifra_simetrico_texto* que albergará todo el desarrollo realizado en este punto de la práctica. En su interior, creo la solución llamada *cifra_descifra_sim_tex.py*. No olvides añadir los **comentarios** que estimes oportunos en el desarrollo del código para una mejor comprensión del mismo.

Para iniciar el desarrollo, incorpora la siguiente sentencia al inicio de la solución. Esta sentencia usará lo explicado anteriormente haciendo uso de Fernet:

```
from cryptography.fernet import Fernet
```

A continuación, **crea una función** llamada `genera_key()`, que sea capaz de **generar la clave** que utilizaremos y almacenarla en un fichero llamado `clave.key`. Para realizarlo utiliza una variable llamada `key` a la que pasarás el método de Fernet `generate_key()`. Y escribe esta contraseña en el fichero creado. Otorga al fichero el modo de escritura (w) y determina que es un fichero binario (b).

Como ejemplo, al ser el primer tratamiento de ficheros que hacemos, se muestra la solución del mismo:

```
def genera_key():
    key = Fernet.generate_key()
    with open("clave.key", "wb") as archivo_clave:
        archivo_clave.write(key)
```

Crea ahora la **función para cargar dicha clave**. En este caso, el nombre de la función será `carga_key()` y lo único que hará será devolver la lectura del fichero de clave a través del método `open()`, ten en cuenta que el método para el acceso será de lectura (r) y habrá que determinar que es un fichero binario (b) porque así lo definimos previamente.

Ahora estaremos en disposición de realizar las tareas principales desde la **función principal** del programa para proceder a encriptar el mensaje y mostrarlo por pantalla. Para ello habrá que:

1. Llamar a la función que genera la clave
2. Crear una variable a la que le pasamos la llamada de la función para cargar la clave
3. Declarar una variable que contendrá un *string* con el mensaje a encriptar (hay que pasarle el método `encode()` para codificar dicho mensaje y que pueda ser tratado por el encriptador). El mensaje será "MENSAJE SECRETO".
4. Iniciar Fernet para que pueda funcionar de forma correcta. Para ello crea una variable llamada `fernet` a la que le pasarás un objeto Fernet con la variable donde has cargado la clave.
5. Realizar, en una nueva variable, el encriptado almacenando el resultado de usar la variable `fernet` creada anteriormente con el método `encrypt()` del mensaje a encriptar.

Ahora, **muestra por pantalla la clave que se ha usado y el mensaje encriptado**.

Abre el fichero `clave.key` para verificar el formato en el que se almacena la clave en el fichero.

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

```
PS C:\Users\Admin\Desktop\PracticaB2.2\Cifrado_Simetrico> python .\cifra_aes.py
Clave usada:  b'aD0syZheYnIFuqhnKmQ4Begu1HEWhLY2Gqzuvj0iUOI='
Mensaje encriptado:  b'gAAAAABm_mery3fRwkMi7NF16LrYsYC8MrTUT_10vcSo41J78PfNbytsJ8gYAs01w0w8YYH7YMQw1HiZQmRkz6udTqs731SjXqb8pQQ4ewS1tBTID6zicY='
PS C:\Users\Admin\Desktop\PracticaB2.2\Cifrado_Simetrico>
```

Como puedes verificar antes de la clave o del mensaje aparece **b'** esto se debe a que se identifica que es un mensaje definido en binario, como habíamos establecido previamente.

Almacena el mensaje encriptado en un fichero llamado `texto_cifrado.bin` para poder verificar que el texto se guarda en el formato correcto. Para ello, deberás crear una nueva función llamada `guarda_msj_encriptado(mensaje_encriptado)` a la que le pasarás como parámetro el mensaje encriptado teniendo en cuenta que será un fichero binario (b). Realiza una llamada a dicha función para verificar el funcionamiento.

Como último paso, comprueba que se corresponde la salida observada en la terminal de VSC con lo que se almacena tanto en el fichero de clave como en el fichero del mensaje encriptado.

En este punto estaremos en disposición de descryptar el mensaje que hemos encriptado haciendo uso de la variable que tenía almacenada dicho mensaje y de la clave simétrica que usamos para su encriptación.

Sigue completando la solución que tenías hasta ahora para ahora hacer uso del método `decrypt()` del objeto Fernet para poder **descryptar el mensaje cifrado**. Almacena el resultado en una variable.

Muestra por la terminal el mensaje inicial descryptado.

Por último, crea una función `guarda_msj_descryptado(msg_descryptado)`, que sea capaz de almacenar en un fichero binario el mensaje en texto plano para que sea legible.

Como conclusión, **verifica el procedimiento** y los pasos que has ido realizando para tener claro cómo funciona este tipo de cifrado y su programación en Python. Verifica que se han creado todos los ficheros de clave y de mensajes que se requieren.

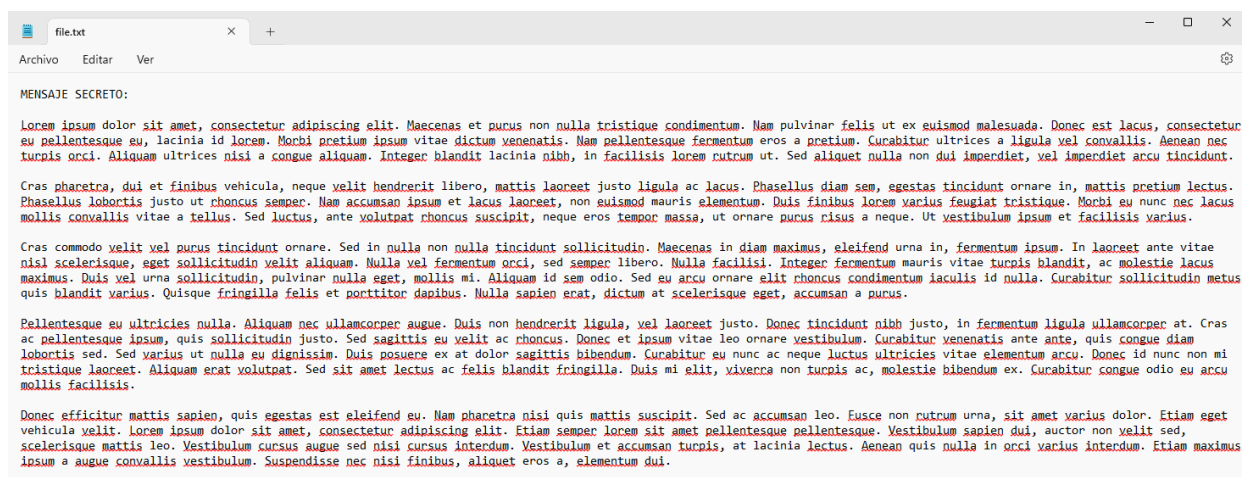
3.2 Cifrado/Descifrado simétrico de archivo

Una vez que hemos cifrado un mensaje haciendo uso de una clave almacenada en un fichero y hemos guardado su contenido en un fichero externo para verificar su funcionalidad, en esta sección realizaremos el cifrado simétrico de un archivo completo para verificar el funcionamiento de este tipo de cifrado atendiendo a archivos completos.

En primer lugar, crea un nuevo directorio raíz llamado `cifra_descifra_simetrico_archivo` en el que almacenarás la solución y todos los ficheros necesarios para la realización de esta parte de la práctica.

Crea una **nueva solución** (con un nuevo directorio llamado `cifra_simetrico_archivo`) en la que crees de nuevo un fichero llamado `cifra_descifra_sim_file.py`, que manejará lo necesario para llevar a cabo la encriptación de un fichero completo.

En primer lugar, debemos crear nuestro fichero de texto (`file.txt`) que será el que encriptemos. Créalo en el directorio raíz de la solución. Accede a <https://es.lipsum.com/>, una web para crear texto de relleno y crea 5 párrafos de texto que deberás copiar y pegar en el fichero `file.txt` añadiendo al inicio la cabecera (MENSAJE SECRETO:)



Este será el fichero que debemos encriptar en nuestra solución.

A continuación, y por no volver a generar la clave, copia de la solución anterior (“Cifrado/Descifrado de mensaje de texto”) el fichero `clave.key` que será el que utilizemos como clave simétrica en esta nueva solución y pégalo en el directorio raíz en el que estes.

En primer lugar, crea una función igual que en la solución anterior para cargar la clave llamada `carga_key()`, ten en cuenta que deberá ser un fichero de lectura y en formato binario.

A continuación, habrá que crear dos funciones para encriptar y desencriptar el fichero. Comenzaremos por la de encriptación haciendo referencia a los siguientes pasos:

1. Crea una función llamada `encriptar()` a la que se le pasará el archivo en texto plano y la clave para poder encriptarlo.
2. Crea un objeto Fernet al que le pases la clave que será el que utilicemos para encriptar el archivo.
3. Lee el fichero `file.txt` que hemos generado anteriormente y almacénalo en una variable (puedes llamarla `archivo_info` ya que almacenará la información del fichero)
4. Crea una variable para guardar la llamada al método `encrypt(archivo_info)`.
5. Crea un nuevo fichero en el que almacenarás la información encriptada que has creado anteriormente. Deberás crearlo como un fichero binario con permisos de escritura. Llama al método `write()` para escribir en él.

Para abordar la función para desencriptar, deberemos seguir estos pasos:

1. Crea una función llamada `desencriptar()` a la que se le pasará el archivo encriptado y la clave para poder desencriptarlo (será la misma al estar en cifrado simétrico).
2. Crea un objeto Fernet al que le pases la clave a utilizar en la desencriptación del fichero.
3. Lee el fichero encriptado y almacena la información leída en una variable.
4. Crea una nueva variable que almacenará la información desencriptada a través del método `decrypt(info_encriptada)`.
5. Crea un nuevo fichero en el que almacenarás la información del fichero desencriptada. Ten en cuenta que deberá ser un fichero con permiso de escritura y formato binario. Al igual que anteriormente, utiliza el método `write()` para escribir en él.

Ahora estaremos en disposición de abordar la función principal del programa, así como las llamadas a las funciones creadas.

En primer lugar, crea una variable en la que almacenes lo que devuelve la llamada a `carga_key()` que nos permitirá tener la clave para poder usar en dicha variable.

Asigna el nombre del fichero en texto plano que tenemos inicialmente (`file.txt`) a una variable, que será la que utilicemos a la hora de realizar llamadas a las funciones correspondientes.

Llama a la función `encriptar()` a la que le pasarás la variable creada anteriormente (nombre del fichero) y la clave cargada.

Crea una variable para almacenar el nombre del fichero encriptado que se pasará a la función `desencriptar()`.

Llama a la función `desencriptar()` pasándole la variable creada (nombre del fichero) y la clave para desencriptar.

Verifica que se crean de forma correcta los ficheros encriptados/desencriptados para comprobar la funcionalidad.

Con esto finalizamos la parte del cifrado simétrico de esta práctica, teniendo a nuestra disposición dos soluciones para cifrar/descifrar de forma simétrica por un lado texto plano en un programa, y por otro ficheros completos.

Confidencialidad: Cifrado Simétrico y Cifrado Asimétrico

Práctica 2.1 - Online

1. Objetivo

El objetivo de esta parte de la práctica será el de asimilar los **conceptos asociados al cifrado asimétrico** ya vistos en la parte de teoría. Para ello, se hará uso del algoritmo de cifrado asimétrico RSA, que nos permitirá, no únicamente encriptar/desencriptar información, sino también poder firmar mensajes, lo que garantizará la autenticidad y confidencialidad de los datos. Se hará uso del software instalado y configurado anteriormente.

2. Cifrado Asimétrico

A diferencia del Cifrado Simétrico estudiado anteriormente, en este caso utilizaremos un Cifrado de tipo Asimétrico a través del **algoritmo RSA**, que como ya sabéis, es un algoritmo criptográfico de clave pública que utiliza factorización de números enteros.

En este caso, tendremos que utilizar una **clave privada y una clave pública** para trabajar. La clave pública podrá ser compartida con quien queramos (agente de confianza o no), miembros que la clave privada deberá ser secreta.

No utilizaremos este algoritmo para cifrar/descifrar información únicamente, sino también para firmar un mensaje con clave privada. Esto nos proporcionará dos casos de uso principales: **autenticación y confidencialidad**. En esta parte de la práctica únicamente atenderemos al cifrado/descifrado (confidencialidad), dejando la parte de la firma para la siguiente práctica.

Seguiremos usando la librería **cryptography** instalada previamente, pero en este caso haremos uso del paquete *Hazmat* (en lugar de Fernet) para llevar a cabo las tareas asociadas al cifrado asimétrico. <https://cryptography.io/en/latest/hazmat/primitives/asymmetric/rsa/>.

2.1 Cifrado/Descifrado asimétrico de texto

Crea un nuevo directorio en el que almacenaras todo lo necesario para crear la solución de esta parte de la práctica. Llama al directorio *cifra_descifra_asimetrico_texto*.

Crea una nueva solución (*cifra_descifra_asim_tex.py*) en la que resolveremos el cifrado/descifrado de información atendiendo al algoritmo RSA (asimétrico).

En primer lugar, importa los objetos **default_backend** y **rsa** del paquete *Hazmat* que nos permitirá usar los métodos necesarios en el desarrollo de la práctica. El primero de ellos, hace referencia a los diferentes tipos de respaldo que ofrece la librería (CipherBackend, RSABackend, etc.), pero siempre usaremos el que viene por defecto. El segundo importará lo necesario para usar el algoritmo RSA en nuestro desarrollo. Para añadirlos a la solución utiliza las siguiente sentencias:

```
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives.asymmetric import rsa
```


A continuación, utilizaremos el método `generate_private_key` para crear nuestra clave asignándole los parámetros que consideremos oportunos. Para ello, crea una variable (`private_key`) a la que, haciendo uso del objeto `rsa` llames al método `generate_private_key()` pasándole los siguientes valores:

- Exponente público (`public_exponent`) = $2^{16}+1 = 65537$ (por defecto siempre usaremos este)
- Tamaño de clave (`key_size`) = 2048
- Backend = `default_backend()`

Con esto tendremos configurada la clave privada que usaremos. En este caso, se atiende a una clave RSA que utiliza un tamaño de 2048 bits.

Para seguir, deberemos crear la **clave pública**. En este caso, crea una variable (`public_key`) a la que asignes la llamada al método `public_key()` haciendo uso de la clave privada generada anteriormente. Con esto, generamos una clave pública a partir de nuestra clave privada.

Podríamos manejar las dos claves guardándolas en un fichero o serializándolas para poder recuperarlas en Hexadecimal, pero no lo haremos para no dilatar la ejecución de la práctica y supondremos que se almacenan de forma correcta si se hace el cifrado y el descifrado correctamente.

Para verificar la existencia de la clave **muestra algunas propiedades** de las claves (privada y pública), como por ejemplo el tamaño (debe ser 2048) o la clase (`__class__`) que devolverá el tipo de objeto que esta almacenado `RSAPrivateKey` o `RSAPublicKey`. Algunas propiedades serán objetos dentro de la definición de la clave y no nos permitirá verlos en texto plano, pero entendemos que son correctos.

Ahora estaremos en disposición de encriptar/desencriptar un mensaje. Para ello, crea una nueva variable que contenga el mensaje a cifrar que deberá ser definido como cadena de bytes (añadiendo "b" al mensaje) para que pueda ser tratado de forma correcta desde los métodos de cifrado/descifrado:

Ejemplo: `mensaje_cifrar = b"Mensaje a cifrar"`

Este será el mensaje que queremos cifrar.

A continuación, crea una nueva variable que contendrá el texto encriptado que será definido haciendo uso del método `encrypt()` de la clave pública (ciframos con clave pública y desciframos con clave privada). Consulta la documentación del método para comprobar que se le pasan 2 parámetros: el mensaje a cifrar (variable creada previamente) y el relleno (`padding`).

En el caso del relleno, habrá que definir el tipo que queremos utilizar pudiendo utilizar OAEP (*Optimal Asymmetric Encryption Padding*), PSS (*Probabilistic Signature Scheme*) o PKCS1v15 (*PublicKey Cryptography Standards*) entre otros. En este caso, haremos uso de OAEP al que habrá que pasarle 3 parámetros: **mgf** que determinará la función de generación de máscara (solo se soporta MGF1), **algorithm**, que será la instancia del algoritmo de hash utilizado y por último, **label** (etiqueta) que determinará la etiqueta a aplicar. Por defecto se establece a valor **None**.

Para facilitar el tratamiento del relleno se propone una posible solución:

```
padding.OAEP(  
    mgf=padding.MGF1(algorithm=hashes.SHA256()),  
    algorithm=hashes.SHA256(),  
    label=None  
)
```


Como vemos, se utiliza un relleno de tipo OAEP en el que atendemos a un algoritmo hash SHA256 (*Secure Hash Algorithm* de 256 bits).

Muestra por pantalla el mensaje cifrado para comprobar que efectivamente se ha producido el cifrado del mismo.

Nos quedará ahora realizar el **descifrado del mensaje cifrado** anteriormente. Para ello crea una nueva variable (*mensaje_descifrado*) a la que le pases el método `decrypt()` usando como objeto la clave privada (usada para descifrar el mensaje en cifrado asimétrico). Al método `decrypt()` se le deben pasar los mismos parámetros que al `encrypt()` en este caso, el mensaje cifrado (ya lo tenemos calculado anteriormente) y el relleno (*padding*) que tendrá los mismos valores que para el caso del cifrado.

Muestra por pantalla el mensaje descifrado para comprobar que coincide con el mensaje inicial que teníamos guardado. Para realizar la verificación, usa una sentencia *if* que sea capaz de comparar ambos mensajes. En caso de que sean iguales se deberá mostrar por pantalla *VERIFICACIÓN CORRECTA* en caso contrario, *VERIFICACIÓN INCORRECTA*.

Incluye en la solución mediante impresiones por pantalla lo que va haciendo la solución para ver los **pasos** que se van dando a la hora de realizar un **cifrado/descifrado asimétrico**.

Un posible ejemplo de ejecución puede ser el siguiente:

```
Creando clave privada...
Clave privada creada
Tamaño Clave Privada: 2048
Clase de Clave Privada: <class 'cryptography.hazmat.bindings._rust.openssl.rsa.RSAPrivateKey'>
Creando clave pública...
Clave privada creada
Tamaño Clave Pública: 2048
Clase de Clave Pública: <class 'cryptography.hazmat.bindings._rust.openssl.rsa.RSAPublicKey'>
Cifrando un mensaje de texto...
Mensaje Cifrado: b'%\xa0\xf9\x96a>ZWIT:\xb41\x84\x8d_c\x96\x86\xf3\xf6\x19\xc0\x02B\xc5\xf4o?\xfd\
f1\xbf\x80\xc3\x89\x1aa+~\xb2p\xec\xce` \xc1\xbc\xb8\x8a<\xf6?{\xbfb3\x9e\x0cg\x18q\x9d\x9f\x8d\x
3)\x9e\xfb\x02=\xad\xd6/\xc4I\xd9\x15?W\x13\xbb\xb8\x84\xfa2\xad\xc1\x1e\x9d\x10\xca\xc7\x152%\x05E
\?x\xc7sUB\x01\xca \xc6g\xb4\xa0\xb5\xc0\x1b\xebM\x01\xda\x16\xcb\x8a\xe4lR\x1Q\x93\xdb\x1f\xcd\
\x8a\x85=\xea\x98f\xe0\xd0\x89\xc6\x1c[\xd9\xaa\x8a\xa5\xce\xf3\xcb\x1a\xbd\xaa\xe8\x83j@\xa5\
d8m\x1c\xf7Lp\xedK\xbb\x87 \xf7\xab\xd1\xa4\xd1\xeb7\xce\xab\xc3L\xb1\xeb\x94\xb4B\xab\xe5\xe2\x08
e4\x18\x1f'
Descifrando un mensaje de texto...
Mensaje Descifrado: b'Mensaje a cifrar'
VERIFICACIÓN CORRECTA
PS C:\Users\Admin\Desktop\PracticaB2.2\Cifrado Asimetrico>
```

2.2 Cifrado/Descifrado asimétrico con *eciespy*

Existen otras librerías que nos pueden permitir realizar tareas de cifrado/descifrado de información haciendo uso de algoritmos asimétricos. En este caso, utilizaremos *eciespy* para llevar a cabo el cifrado/descifrado de un mensaje de texto de forma **muy rápida e intuitiva** ya que nos permite tener funciones directas para poder ver claves, y encriptar/desencriptar información.

Antes de comenzar a desarrollar el desarrollo de esta parte, tendremos que instalar la librería. Para ello utiliza el siguiente comando: `pip install eciespy`. Puedes hacerlo desde la propia terminal de PowerShell que ofrece *Visual Studio Code*.

Una vez finalizada la instalación, crea una nueva solución llamada *cifra_descifra_eciespy.py*. En ella será donde realicemos lo necesario para llevar a cabo el cifrado/descifrado de información.

En primer lugar, importa la librería necesaria para trabajar con *eciespy* para realizar cifrado/descifrado de información.

```
from ecies import encrypt, decrypt
from ecies.utils import generate_eth_key, generate_key
```

Una vez incorporadas, estaremos en disposición de crear lo necesario para poder cifrar/descifrar información.

En primer lugar, crea una **variable** a la que le pases una llamada al **método importado** `generate_eth_key()` esto nos creará directamente las dos claves (privada y pública). Puedes llamar a la variable *claves*.

A continuación, crea dos variables llamadas *clave_privada* y *clave_publica*. Para almacenar el hexadecimal la clave y poder verla bastará con llamar al método `to_hex()` desde la variable que creaste anteriormente para almacenar las claves.

En el caso de la clave pública, deberás hacer uso de esa misma variable, pero accediendo a la clase `public_key` para llamar de nuevo al método `to_hex()` para almacenar dicha variable.

A continuación, **muestra con un *print* las claves privada y pública generadas** para comprobar que son visibles por pantalla en formato hexadecimal.

Una vez que ya tenemos las claves, crea una **variable** que contenga el mensaje que queremos encriptar. En este caso, puedes incluir el texto: “**Mensaje a encriptar**”. Recuerda que sea una cadena de bytes (añade “b” al inicio de la cadena).

Crea una nueva variable que almacenará el mensaje encriptado. Para ello, asigna a dicha variable la llamada al método `encrypt()` al que pasarás la clave pública y el mensaje a encriptar.

Muestra por pantalla el mensaje encriptado. No olvides usar la notación hexadecimal para que se vea de forma correcta (método `hex()`).

Para proceder descryptar el mensaje. Crea una nueva variable que almacenará el mensaje descryptado y utiliza el método `decrypt()` al que pasarás la clave privada y el mensaje encriptado como parámetros.

Muestra por pantalla el mensaje descryptado. En este caso, no hace falta que se pase a hexadecimal ya que tendremos un texto plano y se podrá mostrar sin realizar ninguna conversión.

Al igual que hicimos en la solución anterior, verifica el funcionamiento usando una sentencia *if* que sea capaz de comparar ambos mensajes. En caso de que sean iguales se deberá mostrar por pantalla *VERIFICACIÓN CORRECTA* en caso contrario, *VERIFICACIÓN INCORRECTA*.

Para comprobar si la verificación funciona, **introduciremos un “error”** en la validación y será la creación de otra nueva clave privada para hacer el descryptado de información. Para ello, crea una nueva variable a la que asignarás la llamada al método `generate_key().to_hex()`. Prueba ahora a realizar el descryptado (`decrypt()`) de información atendiendo a esta nueva clave y al mensaje cifrado anteriormente. Almacénalo en una nueva variable. ¿Qué sucede?. Al no estar vinculadas las claves (pública y privada) el mensaje no puede ser descryptado de forma correcta y nos debería saltar el error: `ValueError: MAC check failed`.

Verifica paso a paso lo que va haciendo la solución para comprobar cómo trabaja el cifrado/descifrado simétrico atendiendo a esta librería.

Pese a que hayamos visto dos librerías que nos permiten realizar cifrado/descifrado asimétrico, se recomienda usar la librería de referencia *cryptography*, ya que es la más extendida, la que mayor variedad de opciones nos ofrece y la mayormente soportada por la comunidad.